Git and GitHub user guide

Benjamin Loire

2023-10-27

Contents

0.1 Prerequisites 2

1 GitHub setup **2**
1.1 Setting up GitHub connection 2
1.2 Starting a project 3
1.2.1 From an existing GitHub project 3
1.2.2 From scratch or an existing code repository 4

2 Git **5**
2.1 Updating the local repository 5
2.2 Publish your local changes to the distant repository 6
2.3 Keep track of your work 6
2.4 Moving through the commit history 7
2.5 Branching 8
2.5.1 When to use branches? 8
2.5.2 Merging Branches 8
2.5.3 Rebasing 9
2.5.4 Handling Conflicts 9
2.5.5 The .gitignore file 11

3 GitHub in details **11**
3.1 The content of a GitHub repository 11
3.1.1 Branches 12
3.1.2 Code Link Dropdown 12
3.1.3 Last Commit Information 12
3.1.4 Issues Board 13
3.1.5 Pull Requests 15
3.1.6 GitHub Actions 18
3.1.7 Projects 18
3.1.8 Settings 19
3.2 Afterwords on GitHub 19

4 Troubleshooting **20**
4.1 Push rejected 20
4.1.1 Fixing 20
4.2 Cannot checkout another branch 20
4.2.1 Fixing 20

Note: this is made to be a comprehensive guide to help you get into git. Once you know the name of the commands and roughly what they do, the best resource to keep learning Git is the official documentation.

0.1 Prerequisites

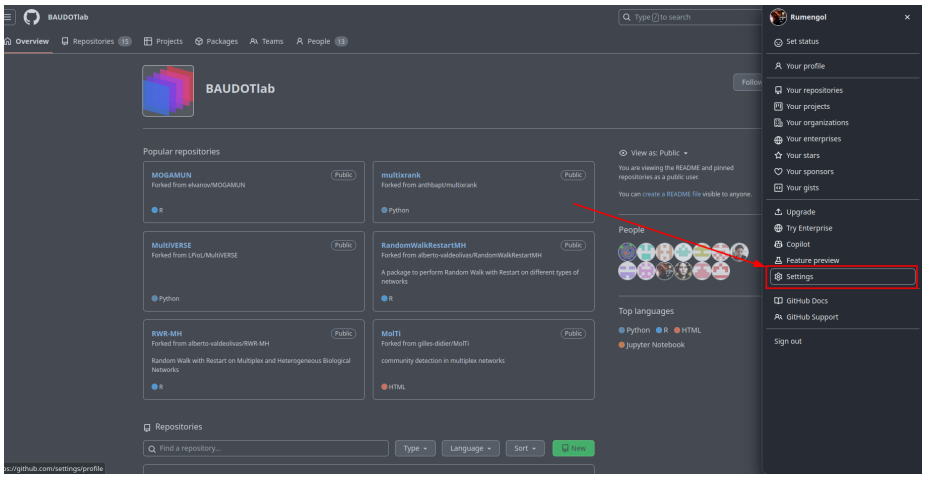
- A GitHub account
- Basic terminal usage knowledge
- Git. It comes preinstalled in some distribution, otherwise go to the official website and download the software.
- A Git terminal (on linux, any terminal is fine. On Windows, you must download git and open a terminal with right click -> "Git Bash Here")

1 GitHub setup

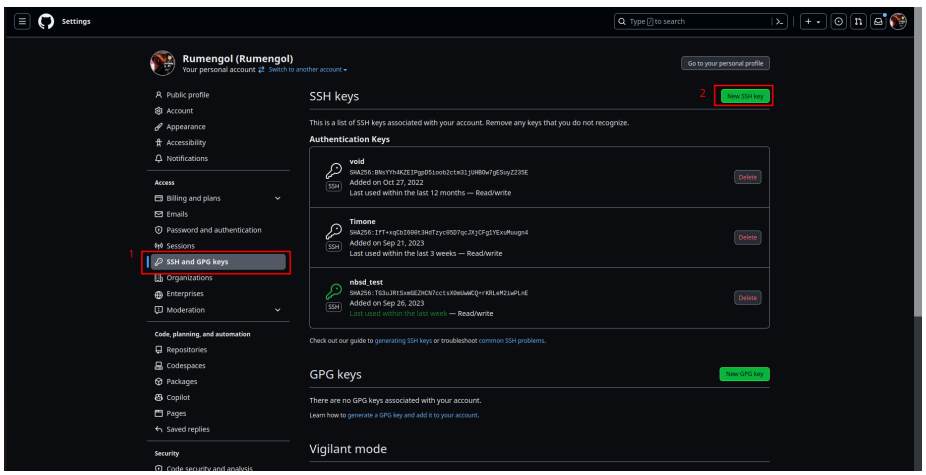
1.1 Setting up GitHub connection

To be able to interact with GitHub, the easiest way is to go through SSH. HTTPS authentification has been dropped in may 2023 and is a hassle to use.

1. In a terminal, type : `ssh-keygen` This will generate the `~/.ssh` folder if it does not exists, and create by default two files inside it: `id_rsa` and `id_rsa.pub`. You can change the file names when prompted, which can be useful if you need multiple ssh keys (for example, `.ssh/github_rsa` and `.ssh/github_rsa.pub`). You can use the same ssh key to connect to as many servers as you wish, but using a different key for each makes it easier to revoke the key later on. You may also set a password for extra security, which will be asked every time you will use this key to connect to anywhere.
2. Copy the content of the public key (by default `id_rsa.pub`) to the clipboard. IMPORTANT : don't copy the private key! It must stay hidden at all times.
3. On the GitHub website, go to `Settings`, then `SSH and GPG keys`.



4. At the top, click on the green `New SSH key`.

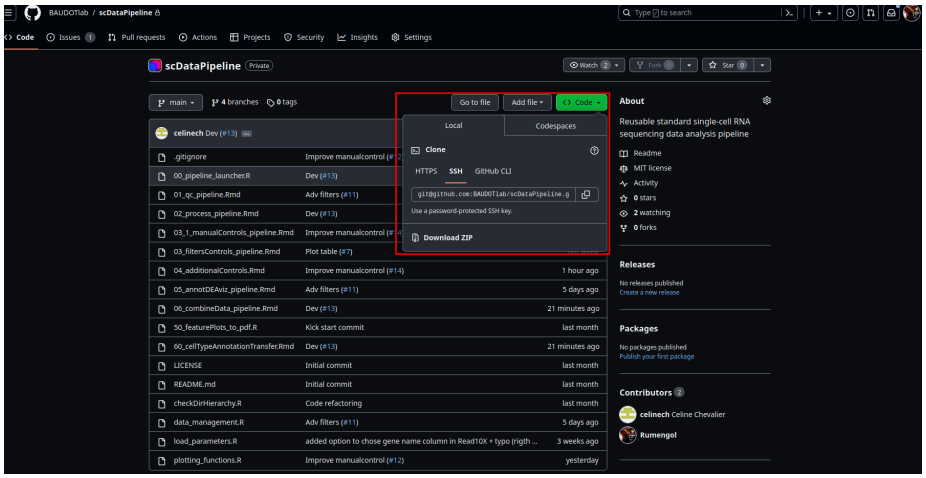


5. Title: A name for your connection to GitHub, usually the name of your computer to remember which key corresponds to which device. Key: Paste here your public key and confirm.

1.2 Starting a project

1.2.1 From an existing GitHub project

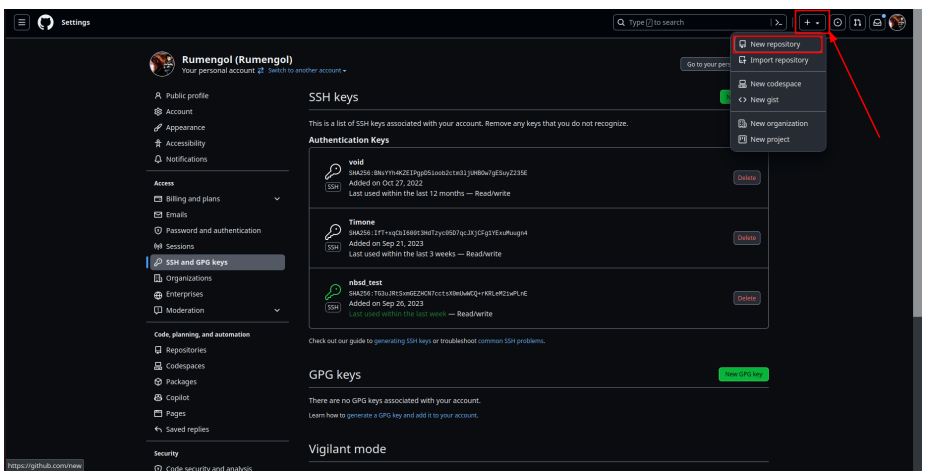
1. Go into the folder that will contain your git project. Note that you will create a new folder here containing your project, so use a general folder, like `~/Documents` or `~/projects` for example.
2. Go to GitHub and click on the green `<> Code` button.
3. In a terminal, type : `git clone [SSH repo URI]` Where the URI is the one who starts with `git@github.com` like here:



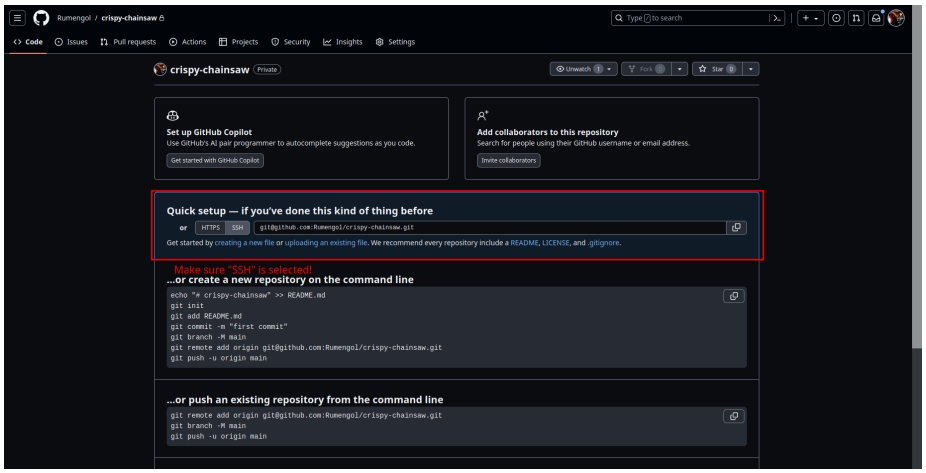
You're all set! Your local repository is now connected with the distant one, you can receive updates and push your own code.

1.2.2 From scratch or an existing code repository

1. Create a folder for your project, for example `~/projects/myproject` and move here (`cd ~/projects/myproject`). Alternatively, go into an existing folder containing your project.
2. Declare the folder as a git repository: `git init`
3. On the GitHub website, create a new repository for your project and get its ssh URI by clicking on the green `<> Code` button like above.



If you left everything unchecked during the repository creation, then the ssh URI is the first line of code on the page.



4. In a terminal, type

```
git remote add origin [SSH repo URI]
```

5. Push the code to the GitHub repository for the first time:

```
git branch -M main
git add .
git commit -m "Initial commit"
git push origin main
```

2 Git

All the commands in this sections are general git commands. They can be used to work with any Git repository, and whenever a distant repository is mentioned, it can be any repository that is not on your local machine. This includes GitHub, of course, but also alternatives like GitLab, SourceForge, and even another computer (though that requires a bit of configuration that will not be covered here).

2.1 Updating the local repository

Make sure on which branch you are (older repositories have `master` as default branch whereas new ones use `main`):

```
git branch
```

Pull the code from the distant repository, assuming you are on the main branch:

```
git pull origin main
```

This reads `git pull [remote name] [remote branch name]`, and will pull the code in the branch you are currently in. It is strongly advised to pull code

from branches with the same name (while it is possible to do something like `git pull origin main` while being on a branch named `master` for example, it may lead to confusion and should be avoided).

You can connect a local branch to a remote one using

```
git pull --set-upstream origin main
```

The `--set-upstream` option will connect your local main branch to the remote origin/main. From now one, using `git pull` and `git pull origin main` from this branch would have the exact same effect. Note that if this branch was created by a `git clone` command, then the remote branch is already connected.

2.2 Publish your local changes to the distant repository

Once you have made changes in the code, you must first commit it. First, you have to explicitly tell git to track those changes.

```
git add .
```

The dot at the end means that all changes made in the current folder and below will be recorded. This include file creation and deletion. If you want to only track certain files for this commit, use:

```
git add [path to file]
```

Then, commit those changes with an explanatory and short name:

```
git commit -m "[commit message]"
```

Good practice: Ideally you must commit your code every time you make a modification, as small as a typo fix or a new function. Realistically, you probably won't. Then, the next best thing is to commit either once you're finished with a specific task, or two times per day: when you go for lunch and for the evening.

Finally, update the distant repository. To avoid the issue of pushing to a branch that had been changed by someone else, remember to pull the code before pushing it.

```
git pull origin main
git push origin main
```

2.3 Keep track of your work

Many commands help you managing your git repository, here are some of them:

```
git status
```

Display information regarding the current state of your working tree: untracked files, unstaged (uncommited) files, as well as how divergent you are from the

upstream branch. It can be helpful in troubleshooting problems, or just to know which files will be included in your next commit.

```
git log
```

Display the list of all your commit history, with the commit messages and hash.

```
git diff
```

Show the changes made between the currently uncommited changes and the last commit. Lines beginning with a minus (-) are lines deleted from the oldest commit and those beginning with a plus (+) are lines added by the newest commit. When a line has been modified, no matter the modification, it will show a deletion immediately followed by an addition.

You can use the commit hash found using `git log` to compare changes between two commits.

```
git diff 7cfa[...] 9fc5[...]
```

2.4 Moving through the commit history

The greatest perk of version control is that it enables you to return to previous versions of the code easily, without losing subsequent changes. Introducing `git checkout`.

But first, let's talk a bit about HEAD. HEAD indicates the position where you are and represents the tip of the currently active tree. If you move to a different branch, the HEAD will be updated to go to the tip of this branch. You can also move in a `detached HEAD` state, which means that any modification you might do will not be saved unless you create a new branch from this commit (see Branching below). This is especially useful to make experimental changes that will be discarded later on.

```
git checkout [commit hash]
```

This command will take you to the specific commit (hashes are unique) in a `detached HEAD` state. You will be able to look at the files from the past and change them, but no save the changes as stated above. When you are done with that, simply type `git checkout [current branch name]` or `git checkout HEAD` to return to your HEAD.

You can also move relatively to where you currently are, using this command and replacing the n with the number of commits you want to go back.

```
git checkout HEAD~n
```

For example, `git checkout HEAD~2` will move you 2 commits before the current commit. HEAD is located at the latest **commit** and not potential uncommitted changes which git is not aware of.

2.5 Branching

A good practice, when working on a collaborative project using git, is to create branches extensively (it is also ideal when working alone, but less critical). Git branches allow multiple people to work on the same project without their changes colliding. A branch is always created **from** another branch and can be merged **into** another branch.

Typically, branches are created from the main or dev branch and merged back into their parent branch once they have achieved their purpose.

2.5.1 When to use branches?

As a rule of thumb, each time you will contribute a new change to a code, whether it is the addition of a feature or a bug fix, you should create a new branch. They are very lightweight, so don't hesitate to make plenty. Since the changes made on one branch are not reflected on the others as long as it is not merged, creating your own branch means you can break things as you like without hindering other's work.

To create a branch, simply type:

```
git branch [branch name]
```

Or to create a branch and move to it in one command:

```
git checkout -b [branch name]
```

2.5.2 Merging Branches

Eventually, you will want to merge the code from multiple branches together. This is where the fun begins. Git provides several methods to do that, and they are all prone to complications. The first of them is the basic `git merge`

```
git merge [branch to merge]
```

To run this command, you must first be on the branch you want the other to be merged **into**. For example, say we want to merge the branch `develop` into `main`:

```
git checkout main
git merge develop
```

If you happen to run into a conflict while merging branches (see Handling Conflicts), you can cancel the operation by running `git merge --abort`. This command will unmerge the branches and reconstruct them as they were before the merge attempt.

Warning: Make sure you don't have uncommitted changes before merging, otherwise the abort command might not work as expected and you may end up losing those changes.

2.5.3 Rebasing

Another way to join two branches together is rebasing. The end result of a rebase and a merge is the same, but the former will make your commit history look nicer and linear. Rather than create a commit which is a fusion of two branches, rebasing moves all the commits in a branch to the tip of another branch.

```
git rebase [branch to rebase ONTO]
```

```
      E---F       develop
     /
A---B---C---D     main
```

Now let's say we rebase `develop` onto `main`:

```
git checkout develop
git rebase main
```

```
              E---F  develop
             /
A---B---C---D        main
```

Now, the git history of `develop` is linear and takes into account all of `main`'s commit. Now, if we want main to be fast-forwarded to the tip of develop to conclude the fusion:

```
git checkout main
git merge develop
```

This command performs a *fast-forward* merge, which is a transparent operation since the commits can't have conflicts. The final git history looks like this:

```
A---B---C---D---E---F   main develop
```

Contrary to a git merge, to rebase you need to be on the branch you want to rebase. This time, we want to rebase the branch `develop` onto `main`:

```
git checkout develop
git rebase main
```

2.5.4 Handling Conflicts

A lot of things in Git may be the cause of the dreaded **merge conflict**. Merging, of course (see Merging Branches), rebasing (see Rebasing), but even pulling from a distant repository (since it involves a merge operation). They look menacing, but are instead a way for you to take decision based on your developer perspective rather than some faulty automation.

A conflict arise when git is trying to merge two commit together and it finds one or multiple lines that have been changed after having diverged. Let's see

how conflicts appear through two examples.

The commit history looks like this:

```
          F---H*    develop
         /     \
A---B---C*--D---E   main
```

Take the line `let x: u32 = 5` which appears in commit A.

If the line is changed in the commit C to be `let x: u32 = 3`, then on the commit H to be `let x: u32 = 7`, git will be able to merge the commit H into E without trouble (assuming there is no other conflict). That is because the commit H already knows that the line changed and since it has been changed a second time, the latest modification takes precedence.

Now, let's look at another commit history:

```
      F---G---H*    develop
     /         \
A---B---C*--D---E   main
```

The code modifications are on the same commits as before, but now develop branched from main *before* commit C. Thus, H has no memory of it and when it is asked to be merged, there is a conflict! Since the two lines have been modified separately, git doesn't know which one should be kept. Is x equal to 3 or 7? Since it cannot take arbitrary decisions, it will ask the developer.

To resolve this conflict, open the file with the conflicting lines. You would see something like that:

```
[code]

<<<<<<< HEAD
let x: u32 = 3
=======
let x: u32 = 7
>>>>>>> develop

[more code]
```

Here, the line `<<<<<<< HEAD` tells that everything under it until the equal line `=======` is code present in the current branch (main) and the line `>>>>>>> develop` tells that everything above it until the equal line `=======` is the code of the branch develop. To resolve this conflict, you have multiple options:

- Keep the changes in MAIN and delete the other line.
- Keep the changes in develop and delete the other line.
- Keep both changes as is.
- Edit the file to make both modifications work together.

In any case, to mark the conflict as resolved, simply remove the conflict indicators: `<<<<<<< HEAD`, `=======` and `>>>>>>> develop`. Repeat for every conflict until your working tree is clear.

Then, run `git merge --continue` or `git rebase --continue` to complete the operation you were doing at first.

2.5.5 The .gitignore file

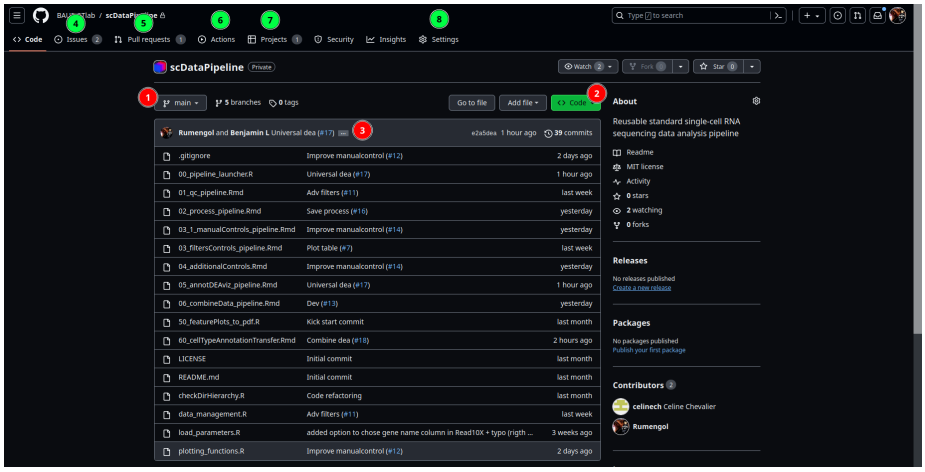
Sometimes, you have files or folders in your git repository that you don't want to keep track of nor push to a remote repository. This can be secret credential, big data or useless log files that have been generated alongside your code. Well, Git got you covered! A very handy tool for that is the `.gitignore` file (don't forget the dot at the beginning).

Create it as a text file in the root of your git repository, and then any file that matches a line of this file will get completely ignored by Git. It won't be tracked, committed or pushed. If a folder is added to this file, everything in this folder will be ignored. The .gitignore works with bash-like patterns, so for example adding a line `*.rds` will exclude every file with the `.rds` extension. Note that you must have exactly one pattern per line.

3 GitHub in details

In the Git section, we saw a bit of the GitHub website. However, the repository interface can be overwhelming at times. We will go over the most important parts using the scDataPipeline repository as our main example.

3.1 The content of a GitHub repository



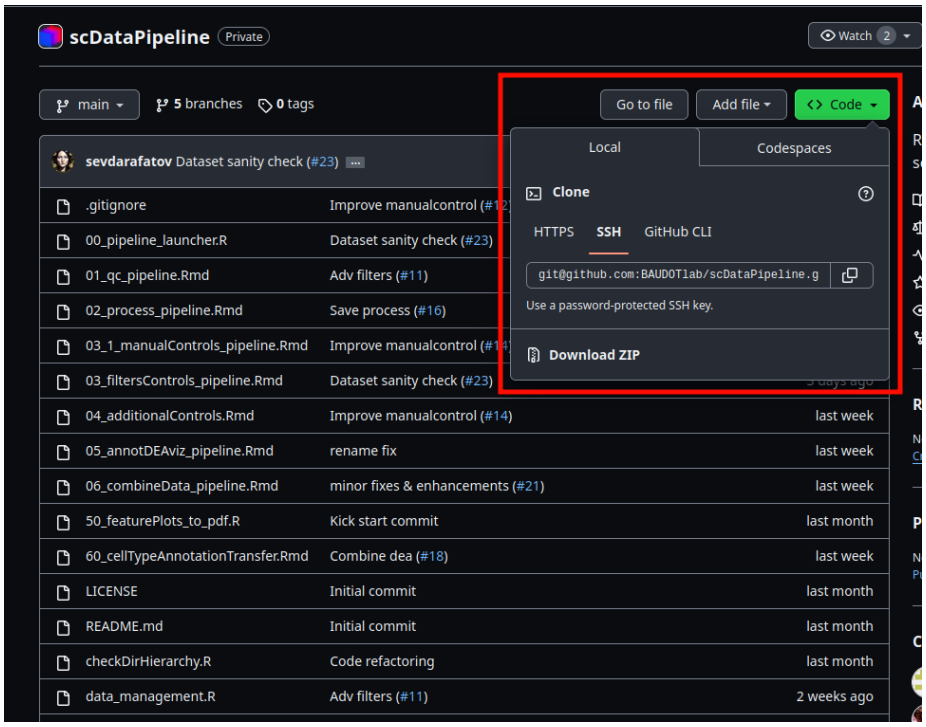
In this view, you can browse the files of the repository, go into the subdirectories

and see the modification history. The repository has three columns: the file names, the name of the last commit that changed this file and how old is that commit.

3.1.1 Branches

Clicking here shows a dropdown list of all the branches that have been pushed to the repository. You can then switch branches at will, for example to browse the code on the website or to pull this specific branch. Note that local branches that have not been pushed will not appear here.

3.1.2 Code Link Dropdown

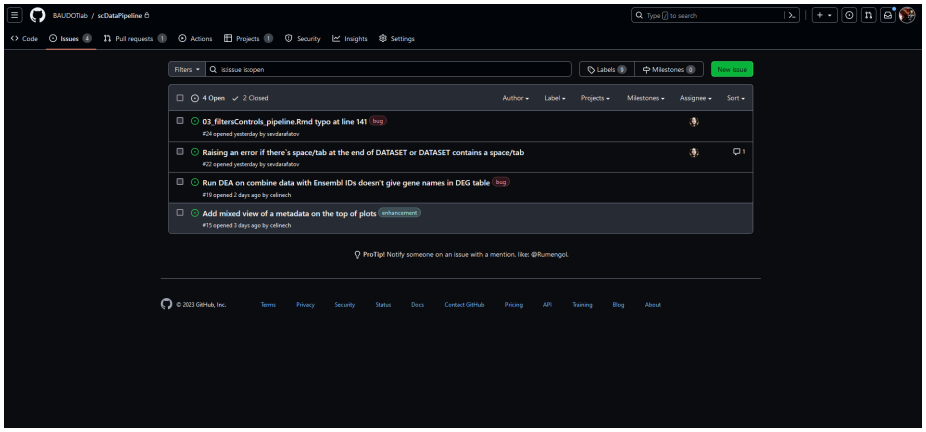


Here you have the repository links. Be sure to select "SSH" before copying the link, as the HTTPS version can be bothersome.

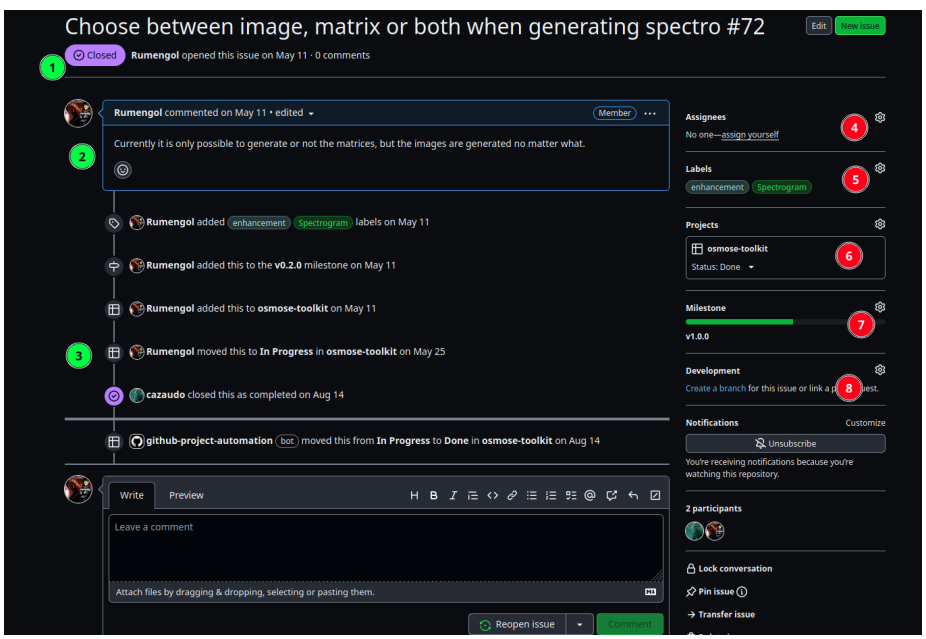
3.1.3 Last Commit Information

This line gives you the name of the latest commit, as well as who pushed it. Clicking on the commit count on the right (here, `39 commits`) takes you to the git history of the branch, where you can browse through previous commits.

3.1.4 Issues Board



The issue board has multiple purposes: bug reporting, feature suggestions and forum. An issue has a title, hopefully meaningful, and a description to explain what it entails. This view gives several information about the issue, and you can sort the issues according to the labels, which can be useful to find issues labelled with `bug` or `high priority` for example.



The issue view show more details about the individual issue, and allow a discussion like a forum. It displays many items, we'll go through each of them:

1. Issue State: Whether it is open or closed. A closed issue is likely to be resolved, or rejected. Open issues are usually actively discussed or worked on.

2. Description: An issue must have at least a title, but although the description is optional, it is highly recommended. Try to provide as much details as possible on the bug, or suggestion, such as context, possible use cases and perhaps a technical proposition.

3. Thread: The thread of an issue shows every actions related to it, such as metadata changes, mentions in other issues/pull requests, state changes, etc… It is also where tother comments will appear, in a chronological top to bottom order.

4. Assignees: Assignees are people designated to work on the issue. It can be anyone working on the project, and can be used as a way to ask someone to focus on a specific issue. Most of the time however, contributors will assign themselves to issues as a way to say that it is worked on and that it is unnecessary for someone else to work on the same thing in parallel. Assignees are a great way to coordinate effort in a team that doesn't meet face to face on a regular basis.

5. Labels: The ultimate sort tool for issues (and anything else, really). In the issue board, it is possible to display only the issues corresponding to a certain label or combinaison. Furthermore, it can highlight the type of change asked for, when the title is not clear enough. You can add as many labels as you want to an issue, just make sure they all are relevant.

GitHub creates by default 9 labels which are general use cases. You can access the list by clicking on the `Labels` button in the issue board screen. The most useful are:

- **bug**, to report a bug or an anormal behavior
- **enhancement**, to suggest a feature or plan future additions
- **question**, which can be used either as a user to ask things about a feature or as a developer to request more details on an issue.

6. Projects: Another way to visualise issues is through the Project view detailed below. In the issue screen, you can add it to an existing project or see its status.

7. Milestone: Milestones are a way to create objectives with deadlines on projects. You can create a milestone in the issue board screen, with just a title, a short description and an optional deadline. Then, issues can be associated to a milestone as a way to set priority and estimate the work needed to reach the next state of the project. Milestones are usually releases or version changes, but it doesn't need to be. Once all the issues associated with a milestone are closed, it is considered reached.

8. Development: If the issue is actively worked on or has a pending pull request associated with it, it will be shown here. It is also a way to kickstart the solving of the issue by directly creating the associated branch.

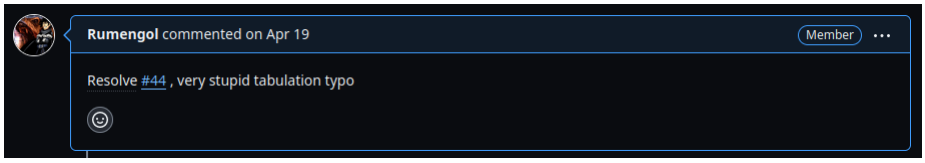
3.1.4.1 Citing an issue

Whether in another issue, a pull request, a code review, or a commit, you can easily refer to existing issues by simply referring to their identifier. Every issue has an incremental identifier shared with pull requests. It is always a # followed by a number. In the Figure 4, it is issue #72. Writing this anywhere will create a link to this issue, and add in the thread where this issue has been mentioned. It is also possible to refer to issues in other repositories by prefixing the repository name, e.g. in this case `Project-OSmOSE/osmose-toolkit#72`

Furthermore, you can automatically close issues by merging pull requests if any of the following terms followed by the reference to the issue is written in the pull request description:

- close
- closes
- closed
- fix
- fixes
- fixed
- resolve
- resolves
- resolved

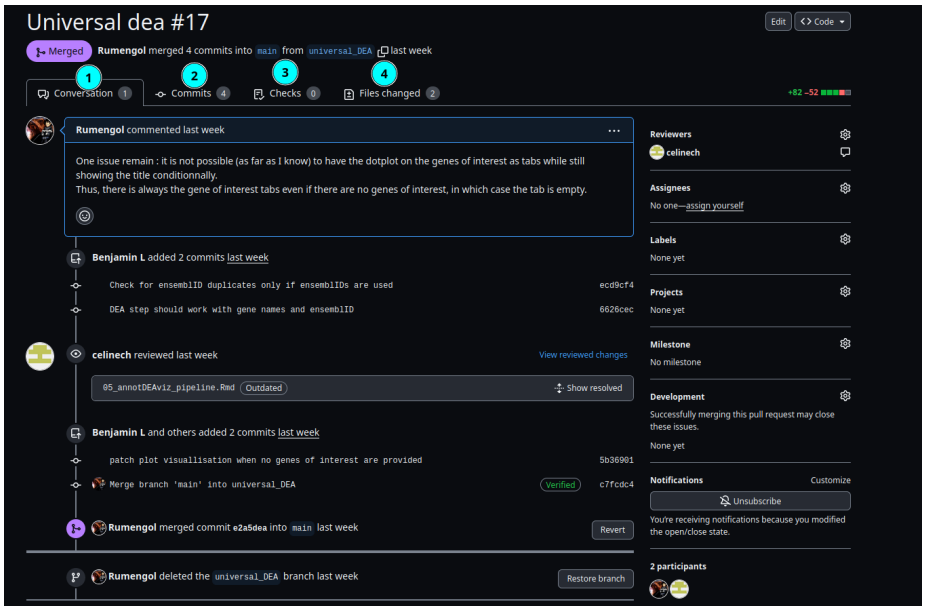
Example of a PR closing an issue upon merge:



3.1.5 Pull Requests

When your branch with its shiny new code has been pushed to the GitHub repository, you still need to merge it with the main codebase. To do that, you need to create a Pull Request (PR). There are multiple ways to do that:

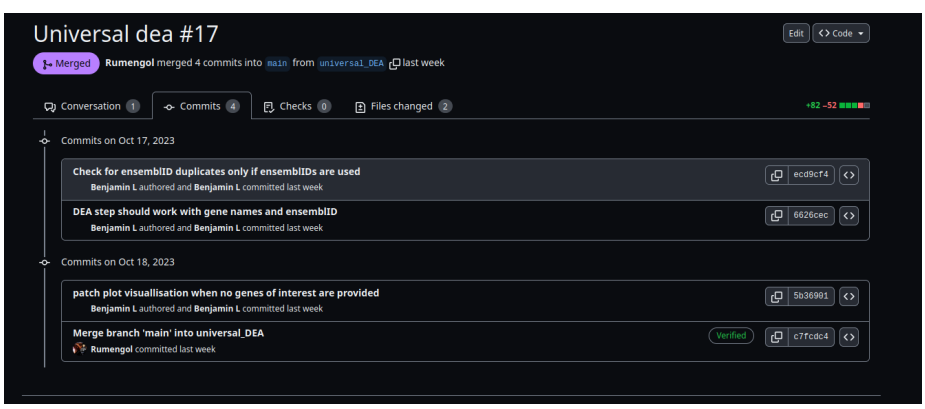
- If you just pushed code to a branch, you might see a banner with the branch name and the button `Compare and Pull Request`.
- In the branch list view, where you can see merged branches and will have a button to do a `New pull request` for unmerged branches.
- In the Pull Requests tab, by clicking on the green `New Pull Request` and selecting the branch you want to merge.



1. Conversation: The main PR screen is very similar with an issue screen. You can see the PR description, a thread with the commit history, chronologically top to bottom, and other contributors can comment, link to issues or other PR, add labels and so on. The main differences are the commit history, which is clickable, and the ability to have multiple discussion: You can answer directly to a comment or a review without adding a whole post.

There is also a list of reviewers above the Assignees. Reviewers are contributors who look at the code added by the PR and decide if it is approved or request changes. All reviewers will be shown here, but it is also possible to request a review from someone using the cogwheel.

2. Commits

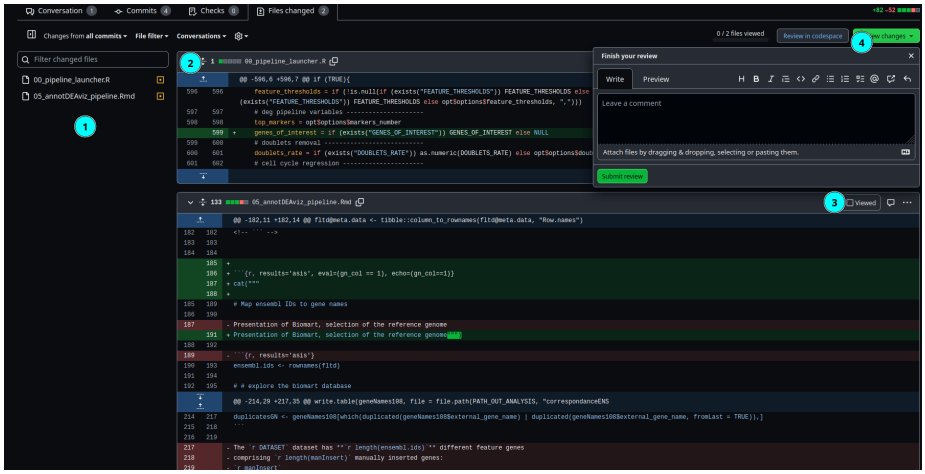


This tab shows the list of commits in a more detailed way, but they are the

exact same shown in the thread.

3. Checks: If the relevant GitHub Actions are added to the repository, their result may appear here. Otherwise, this tab is pretty much irrelevant.

4. Files Changed: Probably the most interesting tab if it is not your PR.



This view is separated in two columns. On the left (1), you have the files that are modified by this PR. A green plus means that the file a new addition, a yellow dot that an existing file was changed and a red cross that an existing file was deleted. Then, the main column (2) shows where the files were modified, showing the changed lines and the three lines above/below. As in git diff, a green line with a plus on the left means that it is added by the PR code, and a red line with a dash on the left means that is was an existing line that has been deleted or changed.

When hovering a line, a blue plus appears, and you can click it to write a comment about this line specifically. Then, you will be given the option to either start a review or leave it as a standalone comment. When reviewing a pull request, you can check the `Viewed` checkbox (3) of a file to mark it as reviewed. Then, when you have seen all the files, you can conclude your review by clicking on `Review changes` **(4)**. You can write a general comment about the review, and when hitting `Submit review`, you will be prompted to put your review in one of three categories: **Comment**, which means you only add a remark on the code, but don't approve nor reject it; **Approve**, which means you completely agree with all the changes of the PR; or **Change Requested** (which must be accompanied by a comment), meaning that the code is not satisfactory and should not be merged as it is.

3.1.5.1 Merging a Pull Request When a Pull Request matches all conditions and is approved, it can be merged with the target branch. To merge a branch, you are given three options:

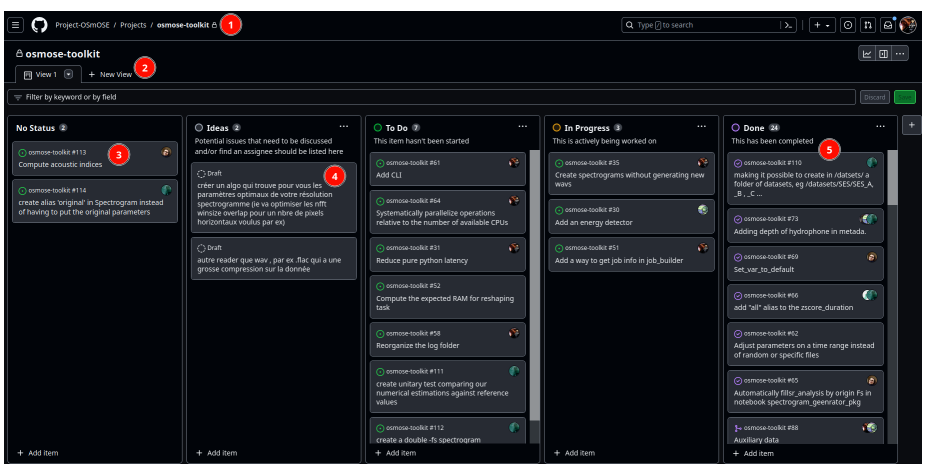
- **(1)** Create a merge commit: Performs a simple `git merge` on the target branch, as it would locally. When many commits are merged this way, the history can become messy.
- **(2)** Squash and merge: A cleaner way to merge. This option first squash all the commits in the branch to merge as one and add it as it was a regular commit to the target branch. In the history, all the branch is reduced to a single commit.
- **(3)** Rebase and merge: Take all the commits in the branch and append them to the target branch, as a `git rebase` would do locally.

3.1.6 GitHub Actions

Actions are a way to add automatized scripts to your GitHub repository. They can be code formatters, continuous deployment/continuous integration (CI/CD), automated unit testing, and even more. Learning how to use them is a different subject altogether, and you probably won't use it unless you are involved in a large scale project.

3.1.7 Projects

Projects are basically another way to visualise issues. You can interact, comment, alter the metadata of an issue in a project, there are however some key differences:



- A project belongs to an **organization**, *not* a single repository **(1)**. This means that multiple repositories can share the same project.
- You can visualise a project in three different ways: either as a **table**, a **board** or a **roadmap** with Gantt diagramm. You can add as many views as you want, as all have their use cases **(2)**. We will focus here on the board view.
- Items in a project can be either *issues* **(3)** or *drafts* **(4)**. The only difference here is that a draft doesn't have all the metadata of an issue and cannot

be commented on. As such, a draft doesn't appear on the issue board.

In a board, you have the possibility to put your items in different categories. By default, those categories represent the state of the item: either an idea that no one is working on at the moment, something that is in progress or completed. Note that if an item is also an issue or a Pull Request, it will be moved to the **Done** category as soon as the issue is closed or the PR is merged **(5)**. The opposite is not true: moving an element to the Done column won't close the associated issue.

There is no constraint on the category names or function. You can set up yours depending on the type of item (single cell, knowledge graph, network…), add more columns (if you choose the default categories, adding a **Priority** column to put all the urgent To Do items might be a good idea), remove those you deem irrelevant… Your call!

3.1.8 Settings

Settings are where you tweak the parameters of your repository. There are a lot of things to explore there, most won't be useful to you, and they are pretty left-explanatory. The most important ones are at the bottom of the **General** tab: the *Danger Zone* . While they are not that dangerous (except for the last option. You probably don't want to delete your repository without a very good reason), they all have a big effect on your repository. This is where you can change your repository from **Public** to **Private**, **Transfer** the ownership to someone else, and of course **Archive** or **Delete** the repository.

If your repository is private, or you want to set other people as administrators, you will be interested in the **Collaborators and teams** tab, where you can add anyone that has a GitHub account. Then, you will have the opportunity to set the permissions for this user. If you are in an organization, you will only be able to set permissions higher than the organization's base role. Otherwise, you can set them using one of the five existing role, **Read** being the least permissive and obviously **Admin** the most. You can read in the dropdown what a role can do to decide which one is best. Having your team members as Maintainers or Admins can be a good start to have people able to manage the repository when you are unavailable, though someone with the **Write** permission can do about everything related to development.

3.2 Afterwords on GitHub

Though GitHub is by far the most popular git platform, it is far from the only one. There are reasons not to use it, such as privacy (the website has been purchased by Microsoft in 2018, and GitHub copilot is known to have used the code of private repositories as training data) or if you want to host the repository on your own server (GitLab among others allows you to do that).

However, GitHub mastery is always a good thing to have, even if you plan on

moving away from it, as many platforms share a lot of high-level concepts, with the main differences being the interface, the philosophy and more specialised features.

4 Troubleshooting

4.1 Push rejected

```
error: failed to push some refs to 'github.com:~~~~'
```

This error likely stems from pushing to a remote branch that has independant changes from your local branch, for example if someone else pushed some code since your last pull.

4.1.1 Fixing

Run `git pull` (resolve potential conflicts), then run `git push` again.

4.2 Cannot checkout another branch

```
error: The following untracked working tree files
would be overwritten by checkout:
```

This shows up when you are trying to move to another branch or commit while having uncommitted changes in the current branch. Git warns you that, should you complete the checkout, some untracked changes would be overwritten and definitely lost, which you proably don't want.

4.2.1 Fixing

Commit your changes with `git add .` and `git commit -m "message"` before running `git checkout` again.